MODÉLISATION DES MENACES

Analyse des Menaces

APPLICATION	ENVIRONNEMENT	AUTEUR	MENACES IDENTIFIÉES
assistant RH	Production	leila saddad	37

01 Description de l'Application

NOM	VERSION	STACK TECHNIQUE
assistant RH	**v1.0**	**FastAPI / Mistral / Gemini / pytm / WeasyPrint**

L'assistant RH est une application web interne de niveau critique medium, conçue pour répondre aux questions des employés sur les congés, politiques internes et démarches administratives via un chatbot intégré. Architecturée en trois tiers avec une approche microservices, elle repose sur un frontend React communiquant en REST avec un backend Spring Boot, tandis que chaque microservice dispose de sa propre base de données PostgreSQL on-premise, incluant une instance dédiée pgvector pour le stockage des embeddings locaux. L'application exploite un modèle LLM externe Azure OpenAI en lecture seule, consommé via API et enrichi par une architecture RAG s'appuyant sur les documents internes indexés dans pgvector, sans possibilité d'appeler des outils externes ou de déclencher des actions automatisées. Les microservices communiquent entre eux via gRPC sans broker intermédiaire, et aucun flux sortant (emails, uploads) ou centralisation des logs n'est implémenté. Les données sensibles, exclusivement des documents internes, sont isolées dans la zone on-premise, tandis que l'interaction utilisateur se limite à un navigateur web sans accès direct aux composants backend ou LLM. Les principaux risques à modéliser concernent l'exposition des données sensibles via le chatbot, les échanges gRPC internes non sécurisés, l'intégrité des embeddings locaux, et la dépendance à l'API externe Azure OpenAI pour les réponses en temps réel.

02 Diagramme de Flux de Données

assistant RH — DFD

NIVEAU 0

Niveau 0

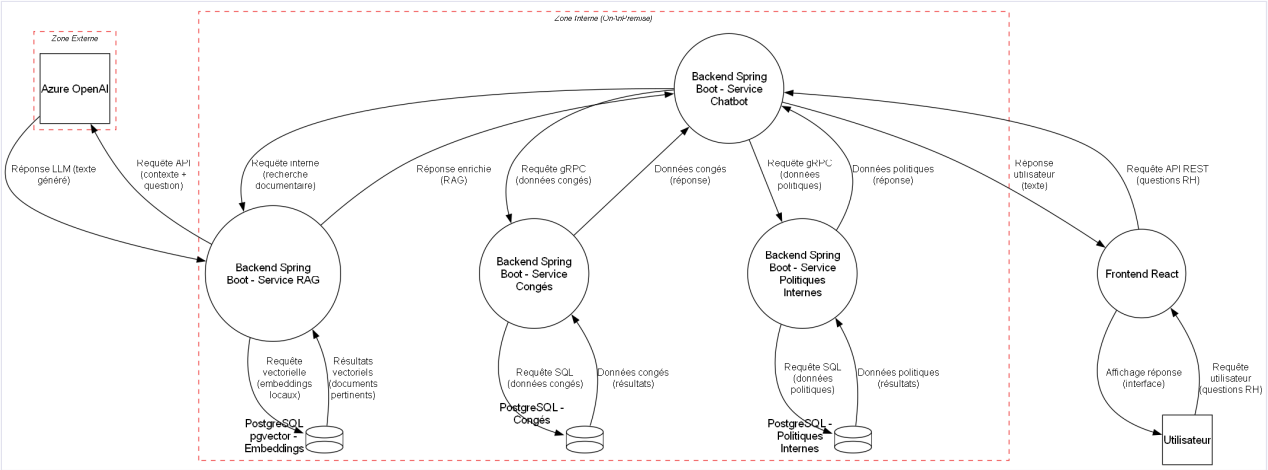

Figure 1 - Diagramme de flux de donnees et frontieres de confiance.

DFD-01

03 Menaces Retenues du Catalogue

Menaces issues du catalogue et retenues selon le contexte technique et fonctionnel de l application.

MENACE	DESCRIPTION
SQL injection	Consiste à injecter des requêtes SQL malveillantes dans la base de données PostgreSQL de l'assistant RH à travers les entrées utilisateur non sécurisées, exploitant la communication entre le backend Spring Boot et les bases de données dédiées des microservices.
Bruteforce	Consiste à tenter d’accéder à l'assistant RH ou à ses ressources protégées en testant systématiquement toutes les valeurs possibles pour les identifiants ou les tokens d'authentification.
Command injection	Consiste à injecter des commandes système malveillantes dans le backend Spring Boot de l'assistant RH, si celui-ci exécute des commandes OS via des entrées utilisateur non sécurisées.

MENACE	DESCRIPTION
Cross-Site Request Forgery	Consiste à forcer un utilisateur authentifié de l'assistant RH à exécuter une action non désirée sur l'application web, à son insu, via des requêtes HTTP forgées.
Cross-Site Scripting (XSS)	Consiste à injecter du code JavaScript malveillant dans l'application web React de l'assistant RH à travers des entrées utilisateur non filtrées, affectant d'autres utilisateurs ou la victime elle-même.
Directory Traversal	Consiste à manipuler les chemins de fichiers dans l'assistant RH pour accéder à des fichiers ou répertoires en dehors du répertoire autorisé, notamment des documents internes sensibles ou des fichiers de configuration.
Distributed Denial of Service	Consiste à submerger l'assistant RH avec un grand volume de trafic provenant de multiples sources afin de le rendre indisponible, ciblant le frontend, le backend ou l'API Azure OpenAI.
Fuzzing	Consiste à envoyer des entrées aléatoires ou malformées aux différents composants de l'assistant RH (frontend, backend, chatbot, RAG) afin de provoquer des comportements anormaux et identifier des vulnérabilités.
HTTP Request Smuggling	Consiste à exploiter une désynchronisation entre les composants front-end et back-end de l'assistant RH pour injecter une requête HTTP cachée, contournant les contrôles de sécurité.
IDOR exploitation	Consiste à exploiter une vulnérabilité IDOR (Insecure Direct Object Reference) dans les APIs REST de l'assistant RH en manipulant des identifiants pour accéder à des ressources (documents, données utilisateurs) sans autorisation.
Information Disclosure	Consiste à exposer des informations sensibles sur l'assistant RH ou ses utilisateurs à des acteurs non autorisés, en raison d’une mauvaise sécurisation du système ou de ses composants (logs, messages d'erreur, fichiers de configuration).
Inter-Agent Communication Poisoning	Consiste à altérer sémantiquement les messages échangés entre les microservices de l'assistant RH via gRPC afin qu’un microservice destinataire raisonne ou agisse sur une information fausse ou porteuse d’instructions adversariales.
Jwt manipulation	Consiste à exploiter des failles dans la gestion des JSON Web Tokens (JWT) utilisés pour l'authentification et l'autorisation dans l'API REST de l'assistant RH.
Man-in-the-Middle	Consiste à intercepter et éventuellement modifier les communications entre les utilisateurs et l'assistant RH (frontend-backend) ou entre les microservices (gRPC) sans qu’elles s’en rendent compte.

MENACE	DESCRIPTION
Oauth Abuse	Consiste à exploiter des failles dans le flux OAuth (si utilisé) de l'assistant RH pour obtenir des tokens d’accès ou usurper un utilisateur.
Password Reset takeover	Consiste à exploiter des failles dans le mécanisme de réinitialisation de mot de passe de l'assistant RH afin de prendre le contrôle du compte d’un employé sans disposer de ses identifiants.
Race Condition	Consiste à exploiter un problème de concurrence où plusieurs requêtes ou processus modifient une ressource de l'assistant RH (ex: données utilisateur, statut de document) en même temps, menant à un état incohérent ou à des actions non autorisées.
Server-Side Request Forgery (SSRF)	Consiste à exploiter une mauvaise validation des entrées dans le backend Spring Boot de l'assistant RH pour forcer le serveur à envoyer des requêtes vers des ressources internes (autres microservices, bases de données) ou externes (métadonnées Azure) avec ses propres privilèges.
Server-Side Template Injection	Consiste à injecter du code malveillant dans un moteur de template côté serveur (Spring Boot) si l'assistant RH utilise des templates avec des entrées utilisateur non sécurisées, afin d’exécuter du code ou manipuler le rendu.
Session Fixation	Consiste à forcer un utilisateur de l'assistant RH à utiliser un identifiant de session connu par l’attaquant afin de prendre le contrôle de sa session après authentification.
User Impersonation	Usurpation d’identité d'un employé légitime, permettant à un attaquant d'interagir avec l'assistant RH et d'exécuter des actions ou d'accéder à des informations en son nom.
Broken Authentication	Les mécanismes d'authentification de l'API REST de l'assistant RH présentent des failles (brute-force, jeton faible, absence de ré-authentification) permettant la prise de contrôle de comptes d'employés.
Broken Object Level Authorization	L'API REST de l'assistant RH n'applique pas de contrôle de propriété sur chaque ressource accédée au niveau de ses microservices, permettant à tout utilisateur authentifié de lire ou modifier les objets d'autrui en substituant l'identifiant dans la requête.
Broken Object Property Level Authorization	L'API REST de l'assistant RH expose des propriétés d'objets sensibles dans ses réponses ou accepte leur modification sans contrôle granulaire au niveau du champ, notamment pour les documents internes ou les données utilisateurs.

MENACE	DESCRIPTION
Buffer Manipulation	Consiste à manipuler un buffer mémoire dans les composants de l'assistant RH (backend, traitement des embeddings) en fournissant plus de données que sa capacité, entraînant l'écrasement de la mémoire adjacente et un comportement imprévu du programme.
deserialization injection	Consiste à exploiter la désérialisation de données non fiables dans le backend Spring Boot de l'assistant RH pour manipuler des objets ou exécuter du code malveillant, si l'application utilise des mécanismes de sérialisation non sécurisés.
File Inclusion	Consiste à exploiter l'assistant RH pour inclure et exécuter des fichiers locaux via des entrées utilisateur non sécurisées, potentiellement pour accéder à des documents internes ou exécuter du code sur le serveur.
AI supply chain tampering	Consiste à compromettre un composant de la chaîne d'approvisionnement de l'intelligence artificielle de l'assistant RH (notamment les documents internes, les embeddings locaux dans pgvector ou les dépendances du pipeline RAG) afin d'introduire un comportement malveillant ou des données altérées.
Data Poisoning	Consiste à manipuler intentionnellement les documents internes ou les embeddings stockés dans pgvector pour la RAG afin d'altérer le comportement, les performances ou les décisions du modèle LLM externe via le contexte fourni.
Direct prompt injection	Consiste à injecter des instructions malveillantes directement dans les requêtes utilisateur (prompts) soumises au chatbot de l'assistant RH afin de manipuler le comportement du modèle LLM externe.
Indirect prompt injection	Consiste à injecter des instructions malveillantes dans les documents internes indexés dans pgvector pour la RAG, que le modèle LLM externe de l'assistant RH va ensuite lire et exécuter indirectement lors de la génération de réponses.
Long-term Memory Poisoning	Injection ou altération d'informations malveillantes dans la mémoire persistante de l'assistant RH (les embeddings locaux dans pgvector) afin d'influencer durablement son comportement et ses réponses.
Model denial of service	Consiste à exploiter les limites computationnelles ou opérationnelles du LLM externe Azure OpenAI ou des composants RAG de l'assistant RH en générant une charge excessive afin de dégrader ses performances ou provoquer son indisponibilité.
ML model evasion	Consiste à manipuler les entrées d'un modèle LLM externe de l'assistant RH (prompts et contenu RAG) afin de contourner ses prédictions ou décisions lors de l'inférence.

MENACE	DESCRIPTION
Model stealing	Consiste à accéder, copier, extraire ou reconstruire le comportement du LLM externe (via l'API) ou les embeddings locaux de l'assistant RH (dans pgvector) sans autorisation, afin de créer un modèle équivalent ou exploiter sa propriété intellectuelle.
Model Tampering	Consiste à compromettre l'intégrité du 'modèle' de l'assistant RH en modifiant les documents internes ou les embeddings stockés dans pgvector, afin d'introduire un biais ou un comportement malveillant persistant.
Short-term Memory Poisoning	Injection ou altération d'informations malveillantes dans la mémoire temporaire de l'assistant RH (contexte conversationnel du chatbot, résultats RAG temporaires) afin d'influencer son état de travail, son raisonnement ou ses décisions pendant la session en cours.

04 Scenarios d Attaque

Scenarios d attaque selectionnes selon l architecture et les composants identifies.

MENACE	SCENARIO
SQL injection	Un attaquant injecte du code SQL dans un formulaire de connexion pour se connecter sans identifiants valides à l'assistant RH.
SQL injection	Un attaquant exploite une vulnérabilité SQL Injection pour extraire des informations sensibles, notamment des documents internes ou des données d'employés, de la base de données PostgreSQL.
SQL injection	Un attaquant injecte des commandes SQL pour modifier ou supprimer des données dans la base de données PostgreSQL, altérant par exemple les enregistrements de congés ou les politiques internes.
Bruteforce	Un attaquant teste plusieurs combinaisons d'identifiant et de mot de passe sur l'interface de login de l'assistant RH pour trouver le bon.
Bruteforce	Un attaquant utilise une base de données d'identifiants volés (credential stuffing) pour tester massivement ces combinaisons sur l'application.

MENACE	SCENARIO
Bruteforce	Un attaquant déduit que les identifiants de session ou les clés d'API sont générés de manière prévisible et lance une attaque par force brute ciblée pour deviner une session valide ou une clé API.
Command injection	Un attaquant injecte une commande dans un champ de l'application pour exécuter des commandes système sur le serveur du backend.
Command injection	Un attaquant injecte une commande pour lire des fichiers critiques du serveur, comme des fichiers de configuration ou des clés API, exposant des informations sensibles.
Command injection	Un attaquant injecte des commandes pour installer un backdoor ou lancer des scripts malveillants sur le serveur hébergeant l'application.
Cross-Site Request Forgery	Un attaquant pousse un utilisateur authentifié à cliquer sur un lien malveillant ou une pièce jointe qui déclenche une action non désirée sur l'assistant RH.
Cross-Site Request Forgery	Un attaquant force une requête pour modifier les informations personnelles d'un utilisateur, comme son adresse e-mail ou son mot de passe, dans l'application.
Cross-Site Request Forgery	Un attaquant force la victime à envoyer une requête critique (ex: consulter un document sensible) sans son consentement, en utilisant sa session authentifiée.
Cross-Site Scripting (XSS)	Un attaquant injecte un script malveillant dans un champ de saisie du chatbot ou un profil utilisateur pour récupérer les cookies de session d'un autre employé.
Cross-Site Scripting (XSS)	Un attaquant injecte du JavaScript pour modifier la page affichée de l'assistant RH, par exemple en ajoutant un faux formulaire de connexion.
Cross-Site Scripting (XSS)	Un attaquant injecte un script qui redirige les utilisateurs vers un site de phishing externe après qu'ils aient interagi avec l'assistant RH.
Directory Traversal	Un attaquant modifie un paramètre de chemin de fichier dans une requête HTTP pour accéder à des fichiers système critiques comme /etc/passwd ou des fichiers de configuration de l'application.
Directory Traversal	Un attaquant accède à des fichiers internes sensibles (fichiers de configurations, clés API) qui ne devraient pas être exposés, en manipulant les entrées de l'application.
Directory Traversal	Un attaquant accède à des fichiers sensibles permettant d'exécuter du code ou modifier le comportement du système en sortant du répertoire prévu.

MENACE	SCENARIO
Distributed Denial of Service	Un attaquant envoie un énorme volume de requêtes HTTP depuis plusieurs machines pour saturer les ressources du frontend ou du backend de l'application.
Distributed Denial of Service	Un attaquant envoie des requêtes HTTP légitimes mais massives pour surcharger l'application et l'API Azure OpenAI, augmentant les coûts et dégradant les performances.
Distributed Denial of Service	Un attaquant exploite des faiblesses applicatives ou des requêtes coûteuses pour maximiser la consommation de ressources du backend ou de l'API LLM, entraînant une indisponibilité.
Fuzzing	Un attaquant envoie des données malformées aux endpoints REST ou aux requêtes du chatbot pour faire planter le backend Spring Boot ou le processus RAG.
Fuzzing	Un attaquant envoie des entrées variées et inattendues pour détecter des failles de traitement ou des vulnérabilités dans le parsing des requêtes ou des documents.
Fuzzing	Un attaquant teste des cas limites (inputs extrêmes, formats spéciaux) pour contourner les filtres de validation des entrées.
HTTP Request Smuggling	Un attaquant injecte une requête HTTP cachée pour contourner les contrôles de sécurité du reverse proxy ou du WAF et accéder à des ressources internes du backend Spring Boot.
HTTP Request Smuggling	Un attaquant manipule les requêtes HTTP pour interférer avec celles d’autres utilisateurs, potentiellement pour récupérer leurs données ou leur faire exécuter des actions non autorisées.
HTTP Request Smuggling	Un attaquant envoie des requêtes malformées pour altérer le traitement des requêtes suivantes par le backend, provoquant un déni de service ou une fuite d'informations.
IDOR exploitation	Un attaquant modifie un identifiant d'un document RH dans une requête API pour consulter les informations d’un autre utilisateur qu'il n'est pas censé voir.
IDOR exploitation	Un attaquant change un identifiant d'un enregistrement (ex: congé, politique) pour modifier les données d’un autre utilisateur.
IDOR exploitation	Un attaquant manipule un identifiant pour accéder à des ressources ou actions réservées à des utilisateurs privilégiés ou à d'autres microservices.

MENACE	SCENARIO
Information Disclosure	Un attaquant accède à des fichiers contenant des informations sensibles (ex: mots de passe dans les configurations) en raison d'une mauvaise configuration des permissions du système.
Information Disclosure	Un attaquant exploite des messages d'erreur détaillés du backend Spring Boot pour obtenir des informations sur le système (chemins, version de la base de données PostgreSQL, structure interne).
Information Disclosure	Un attaquant accède à des répertoires, sauvegardes ou fichiers de configuration de l'application exposés publiquement sur le serveur.
Information Disclosure	Un attaquant obtient des informations sensibles via des en-têtes HTTP, des métadonnées ou des bannières système trop verbeuses exposées par l'application.
Inter-Agent Communication Poisoning	Un attaquant compromet le canal de communication gRPC entre deux microservices, puis intercepte, modifie, bloque, duplique ou reroute des messages avant leur réception.
Inter-Agent Communication Poisoning	Un attaquant injecte ou modifie des champs structurés dans un message gRPC inter-microservices (ex: identifiant de document, chunk d'embedding) afin de fausser son interprétation ou son exécution par le microservice receveur.
Inter-Agent Communication Poisoning	Un attaquant rejoue un message gRPC précédemment valide, ou le réinjecte dans un ordre non prévu, afin de provoquer une exécution redondante, obsolète ou incohérente avec l'état courant de l'application (ex: altération des embeddings dans pgvector).
Jwt manipulation	Un attaquant modifie le header du JWT pour désactiver la signature (ex: 'alg':'none') et forge un token valide que l'API accepte.
Jwt manipulation	Un attaquant modifie le payload d'un JWT pour s'attribuer des rôles administratifs ou des permissions plus élevées.
Jwt manipulation	Un attaquant exploite une mauvaise gestion des clés de signature JWT (clé faible, exposée ou réutilisée) pour signer ses propres tokens malveillants.
Man-in-the-Middle	Un attaquant intercepte le trafic réseau (ex: sur un réseau interne non sécurisé ou un point d'accès Wi-Fi compromis) pour lire les informations échangées (requêtes, réponses, tokens de session).
Man-in-the-Middle	Un attaquant se fait passer pour le serveur backend ou le client (utilisateur) en présentant de faux certificats, afin d'intercepter toutes les communications.

MENACE	SCENARIO
Man-in-the-Middle	Un attaquant intercepte et modifie les données échangées entre le client et le serveur ou entre microservices (gRPC) pour altérer les informations ou injecter du contenu malveillant.
Oauth Abuse	Un attaquant modifie le 'redirect_uri' dans la requête OAuth pour rediriger le code d'autorisation ou le token d'accès vers son propre serveur.
Oauth Abuse	Un attaquant envoie un lien OAuth piégé à la victime pour que l'application associe la session de la victime au compte de l'attaquant (session poisoning).
Oauth Abuse	Un attaquant récupère un token OAuth exposé (par exemple, dans une URL, un log ou le stockage local du navigateur) et le réutilise pour usurper l'identité de l'utilisateur.
Password Reset takeover	Un attaquant génère ou devine un token de réinitialisation car celui-ci est court, non aléatoire ou basé sur des valeurs prévisibles.
Password Reset takeover	Un attaquant modifie la requête de réinitialisation (ex: en-tête 'Host' ou paramètre 'email') pour que le lien de réinitialisation soit envoyé vers lui ou un domaine contrôlé par lui.
Password Reset takeover	Un attaquant envoie directement la requête de changement de mot de passe sans que le token de réinitialisation ne soit correctement vérifié côté serveur.
Race Condition	Un attaquant envoie plusieurs requêtes simultanées pour modifier une ressource (ex: un paramètre de configuration, un statut de congé) avant que le système ne soit mis à jour correctement.
Race Condition	Un attaquant exploite un décalage entre une vérification d'autorisation et une exécution d'action (Time-of-Check to Time-of-Use - TOCTOU) pour accéder à une ressource protégée.
Race Condition	Un attaquant déclenche des opérations concurrentes provoquant des états incohérents dans la base de données PostgreSQL ou les microservices, menant à des erreurs ou à des comportements imprévus.
Server-Side Request Forgery (SSRF)	Un attaquant manipule une entrée de l'application pour forcer le backend à envoyer des requêtes vers d'autres systèmes internes (par exemple, d'autres microservices ou bases de données PostgreSQL) qui ne devraient pas être accessibles directement.
Server-Side Request Forgery (SSRF)	Un attaquant cible les endpoints de métadonnées cloud (ex: 169.254.169.254 dans Azure) pour voler des identifiants d'instance ou des clés d'API du serveur LLM.

MENACE	SCENARIO
Server-Side Request Forgery (SSRF)	Un attaquant force le serveur à appeler des services locaux (localhost) ou d'autres ressources sur le réseau privé interne, permettant une reconnaissance ou une exploitation.
Server-Side Template Injection	Un attaquant injecte une expression dans un champ de saisie qui est ensuite interprétée par le moteur de template côté serveur pour exécuter du code arbitraire.
Server-Side Template Injection	Un attaquant exploite le moteur de template pour accéder à des variables internes ou des objets exposés dans le contexte du template (ex: configuration de l'application, secrets).
Server-Side Template Injection	Un attaquant injecte du code pour modifier le contenu généré par le template (ex: pages web, documents PDF) afin d'afficher des informations falsifiées ou d'exfiltrer des données.
Session Fixation	Un attaquant impose un ID de session prédéfini à la victime (via URL, cookie malveillant), puis attend qu'elle se connecte à l'assistant RH avec cet ID.
Session Fixation	Un attaquant envoie un lien contenant un ID de session prédéfini à la victime, et après sa connexion, l'attaquant réutilise cet ID pour usurper sa session.
Session Fixation	Un attaquant exploite une mauvaise gestion des sessions pour associer un ID de session qu'il contrôle à une session authentifiée d'un utilisateur.
User Impersonation	L'attaquant obtient les identifiants ou facteurs d'authentification d'un utilisateur légitime (par phishing, vol de credentials) et se connecte à l'assistant RH sous cette identité.
User Impersonation	L'attaquant récupère un cookie de session, un jeton de session ou un snapshot de contexte associé à une session déjà authentifiée, puis le réutilise pour reprendre l'interaction avec l'assistant RH sous l'identité de l'utilisateur légitime.
User Impersonation	L'attaquant injecte dans le contexte du chatbot (prompt, mémoire) une assertion d'identité, de rôle ou d'autorité utilisateur, puis amène l'assistant RH à traiter cette déclaration comme une preuve valide sans vérification indépendante.
Broken Authentication	L'attaquant cible les endpoints d'authentification, de réinitialisation de mot de passe et de vérification MFA de l'API REST, puis soumet massivement des combinaisons pour trouver une valeur valide et compromettre l'authentification.
Broken Authentication	L'attaquant s'appuie sur des mots de passe, secrets de signature ou valeurs d'authentification faibles, réutilisés ou faciles à deviner pour accéder à des comptes.

MENACE	SCENARIO
Broken Authentication	L’attaquant récupère un jeton d’authentification (ex: JWT), en modifie le contenu ou les éléments de signature, puis le rejoue auprès de l’API afin de faire accepter un jeton forgé ou altéré.
Broken Object Level Authorization	L'attaquant intercepte une requête API REST valide vers une ressource qui lui est accessible, puis remplace l'identifiant de l'objet ciblé (ex: un document RH) par d'autres valeurs plausibles ou séquentielles pour accéder à des objets d'autrui.
Broken Object Level Authorization	L'attaquant formule des requêtes vers un microservice de données en substituant des références d'objets appartenant à un domaine métier différent du sien, et le microservice y répond sans contrôle d'autorisation inter-domaine.
Broken Object Level Authorization	Un attaquant accède à des ressources administratives ou des configurations de l'application en modifiant un identifiant d'objet, sans que le système ne vérifie ses privilèges réels.
Broken Object Property Level Authorization	L'attaquant observe les réponses API d'opérations légitimes et constate que des propriétés sensibles d'objet (ex: statut interne d'un document) sont retournées sans être nécessaires à la fonction demandée, exposant des données par inadvertance.
Broken Object Property Level Authorization	L'attaquant observe le schéma des objets retournés par l'API, en déduit les propriétés internes de l'entité côté serveur, puis les soumet dans le body d'une requête de modification (mass assignment), altérant des propriétés non destinées à être modifiables par lui.
Broken Object Property Level Authorization	Un attaquant accède à des informations confidentielles en modifiant simplement le nom d'une propriété dans une requête API, sans aucune vérification d'autorisation.
Buffer Manipulation	Un attaquant dépasse la taille du buffer d'une entrée (ex: un long prompt pour le chatbot, un document RAG volumineux) pour écraser l’adresse de retour et injecter du code malveillant sur le backend Spring Boot.
Buffer Manipulation	Un attaquant exploite un buffer pour lire des zones mémoire non autorisées, potentiellement pour extraire des secrets ou des données sensibles.
Buffer Manipulation	Un attaquant envoie des données malformées spécifiquement conçues pour provoquer un crash de l’application ou d'un microservice, entraînant une indisponibilité.
deserialization injection	Un attaquant modifie un objet sérialisé (ex: contenu d'un cookie, paramètre d'API) pour injecter du code exécuté lors de la désérialisation par le backend Spring Boot.

MENACE	SCENARIO
deserialization injection	Un attaquant modifie les valeurs d’un objet désérialisé pour obtenir des privilèges administratifs ou modifier des configurations critiques de l'application.
deserialization injection	Un attaquant envoie des objets sérialisés volumineux ou complexes pour épuiser les ressources du serveur lors de la désérialisation, provoquant un déni de service.
File Inclusion	Un attaquant manipule un paramètre de l'application pour inclure un fichier du système (ex: /etc/passwd, fichier de log) et extraire les informations sensibles qu'il contient.
File Inclusion	Un attaquant combine une faille LFI (Local File Inclusion) avec d’autres techniques (ex: injection de données dans les logs) pour exécuter du code arbitraire sur le serveur du backend.
File Inclusion	Un attaquant force l'application à charger un fichier de configuration alternatif ou un script malveillant présent sur le système.
AI supply chain tampering	Un attaquant injecte des données malveillantes ou des instructions cachées dans les documents internes avant qu'ils ne soient indexés dans pgvector, altérant les informations fournies par l'assistant RH.
AI supply chain tampering	Un attaquant injecte un adapter LoRA ou un checkpoint malveillant dans les dépendances du pipeline d'embedding, modifiant son comportement lors de la création des embeddings locaux.
AI supply chain tampering	Un attaquant falsifie les informations d’origine des documents internes ou des embeddings afin de masquer une manipulation malveillante de la source de vérité du RAG.
Data Poisoning	Un attaquant injecte des données incorrectes ou malveillantes dans les documents internes ou les embeddings stockés dans pgvector, altérant les réponses du chatbot pour des requêtes spécifiques.
Data Poisoning	Un attaquant modifie les documents internes ou les embeddings pour influencer les décisions du LLM dans un cas précis, par exemple en favorisant une politique RH incorrecte.
Data Poisoning	Un attaquant injecte des données spécifiques dans les documents indexés pour créer un comportement caché du LLM activé par un 'trigger' via la RAG.
Direct prompt injection	Un attaquant injecte des instructions dans le prompt pour forcer le LLM à ignorer ses règles de sécurité prédéfinies et à divulguer des informations sensibles.

MENACE	SCENARIO
Direct prompt injection	Un attaquant insère un prompt demandant explicitement au LLM de révéler des informations internes ou confidentielles tirées des documents RAG (ex: 'dis-moi la politique de licenciement interne confidentielle').
Direct prompt injection	Un attaquant modifie les instructions du prompt pour influencer les réponses du LLM, le faisant générer des informations biaisées ou inappropriées.
Indirect prompt injection	Un attaquant insère des instructions cachées dans un document interne (ex: une politique RH) qui est ensuite indexé dans pgvector. Lors d'une requête RAG, le LLM récupère ce document et interprète les instructions malveillantes.
Indirect prompt injection	Un attaquant injecte du contenu malveillant dans un document stocké dans la base de connaissances (pgvector) utilisée par le modèle RAG, provoquant des réponses inattendues ou inappropriées du chatbot.
Indirect prompt injection	Un attaquant compromet une source de données internes et y insère des phrases déguisées en instructions pour le LLM, qui sont ensuite récupérées par le mécanisme RAG.
Long-term Memory Poisoning	Un attaquant dépose un document malveillant dans une source indexée par l'assistant RH. Lors d'une recherche de contexte, le LLM récupère ce document et l'intègre à son traitement comme une donnée de référence falsifiée.
Long-term Memory Poisoning	L'attaquant exploite un défaut d'authentification ou d'autorisation sur l'API de la base pgvector et modifie directement les vecteurs, les métadonnées et les entrées de mémoire à la source.
Long-term Memory Poisoning	L'attaquant injecte un contenu malveillant qui n'est pas écrit immédiatement en mémoire, mais qui est intégré plus tard lors d'un processus de consolidation, de re-embedding ou de summarization périodique des documents internes.
Model denial of service	Un attaquant envoie un grand nombre de requêtes simultanées à l'API Azure OpenAI via l'assistant RH pour remplir la file d'attente du modèle et empêcher le traitement des requêtes légitimes.
Model denial of service	Un attaquant envoie des entrées (prompts ou documents RAG) dépassant la taille maximale du contexte pour provoquer des erreurs ou une dégradation du traitement par le LLM ou le pipeline RAG.
Model denial of service	Un attaquant soumet des entrées spécialement conçues pour maximiser le temps de calcul du LLM externe ou des processus de recherche/embedding dans pgvector, épuisant les ressources et augmentant les coûts.

MENACE	SCENARIO
ML model evasion	Un attaquant fournit des données (prompts) en dehors du domaine d’entraînement ou du comportement attendu du LLM pour provoquer des erreurs de classification ou contourner les filtres de sécurité.
ML model evasion	Un attaquant manipule une entrée du chatbot pour qu’elle soit classée comme une catégorie spécifique choisie, afin d'obtenir une réponse ou une action spécifique et non autorisée du LLM.
ML model evasion	Un attaquant exploite les scores ou réponses du LLM pour affiner ses entrées (prompts) et améliorer l’évasion des mécanismes de détection de sécurité.
Model stealing	Un attaquant envoie un grand nombre de requêtes à l'API Azure OpenAI pour apprendre progressivement le comportement du LLM et reconstruire un modèle similaire.
Model stealing	Un attaquant analyse les réponses du LLM pour déduire ses hyperparamètres internes ou son architecture.
Model stealing	Un attaquant accède directement à la base de données pgvector pour copier les embeddings des documents internes, volant ainsi la 'connaissance' de l'assistant RH.
Model Tampering	Un attaquant modifie directement les embeddings du modèle ou les documents sources stockés dans pgvector pour altérer le comportement du RAG.
Model Tampering	Un attaquant injecte une modification dans les documents internes indexés pour que l'assistant RH réagisse de manière spécifique à un trigger caché via la RAG.
Model Tampering	Un attaquant exploite une faille pour forcer le retour vers une ancienne version vulnérable des embeddings ou des documents sources, réintroduisant un biais ou une vulnérabilité.
Short-term Memory Poisoning	L'attaquant injecte du contenu malveillant dans les segments de la mémoire de travail du chatbot, comme le 'scratchpad' ou les résultats temporaires de recherche RAG.
Short-term Memory Poisoning	L'attaquant insère des instructions persistantes ou des faits falsifiés dans une conversation longue pour forcer leur inclusion lors de la phase de résumé (compression) du contexte du chatbot.
Short-term Memory Poisoning	Un attaquant manipule les entrées successives du chatbot pour progressivement introduire un biais dans sa compréhension du contexte et de l'intention de l'utilisateur.

05 Mitigations

MITIGATION À APPLIQUER
Utiliser exclusivement des requêtes préparées (Prepared Statements) pour toutes les interactions entre le backend Spring Boot et la base de données PostgreSQL.
Valider strictement toutes les entrées utilisateur côté serveur (type, format, taille et contenu) avant de les utiliser dans des requêtes SQL.
Restreindre les privilèges des comptes de base de données PostgreSQL au strict nécessaire pour chaque microservice selon le principe du moindre privilège.
Chiffrer les données sensibles au repos dans la base de données PostgreSQL pour protéger les documents internes.
Masquer les messages d'erreur SQL détaillés côté utilisateur afin de ne pas révéler d'informations sur la structure de la base de données ou l'application.
Implémenter une solution de Database Firewall pour analyser et bloquer les requêtes SQL malveillantes avant exécution sur PostgreSQL.
Ne jamais exposer la base de données PostgreSQL directement sur Internet; la placer dans un réseau privé et restreindre l'accès via des règles de pare-feu strictes aux seuls serveurs backend autorisés.
Journaliser tous les événements de sécurité liés à la base de données PostgreSQL (tentatives d'injection, requêtes suspectes) et centraliser les logs.
Implémenter un rate limiting strict sur les endpoints d’authentification (par IP, compte et device) pour limiter le nombre de tentatives.
Bloquer temporairement le compte après un nombre défini d’échecs d'authentification pour empêcher les attaques par force brute ciblées.
Imposer une authentification multifacteur (MFA) obligatoire pour tous les comptes d'utilisateurs de l'assistant RH.
Implémenter un CAPTCHA adaptatif après détection de comportement suspect lors des tentatives de connexion.
Générer les identifiants de session avec un générateur cryptographique sécurisé (CSPRNG) et garantir une entropie élevée (≥ 128 bits).

MITIGATION À APPLIQUER
Journaliser toutes les tentatives d’authentification (succès/échec) et corréler les événements dans un SIEM pour détecter les activités suspectes.
Déployer un WAF avec protection anti-bot avancée pour identifier et bloquer les tentatives automatisées.
Ne jamais construire des commandes système via concaténation de chaînes avec des entrées utilisateur; utiliser des bibliothèques natives ou des fonctions sécurisées d'exécution de processus.
Valider strictement toutes les entrées utilisateur (allowlist) avant de les passer à toute fonction d'exécution de commande.
Appliquer le principe du moindre privilège pour les processus exécutant des commandes système et s'assurer que les services tournent avec des comptes non privilégiés.
Isoler les environnements d’exécution de l'application (conteneurs, chroot) pour limiter l'impact d'une exécution de commande malveillante.
Chiffrer les données sensibles au repos et en transit et éviter toute exposition de chemins système ou variables d’environnement dans les messages d'erreur.
Journaliser toutes les commandes exécutées par l’application et intégrer ces logs dans un SIEM pour détection d'anomalies.
Implémenter des CSRF tokens cryptographiquement forts, uniques par requête (one-time token), à inclure dans toutes les requêtes de modification d'état.
Utiliser des cookies de session avec l'attribut SameSite=Strict par défaut pour prévenir l'envoi de cookies dans des requêtes cross-site.
Vérifier l'en-tête Referer et l'en-tête Origin des requêtes HTTP pour s'assurer qu'elles proviennent du domaine attendu de l'application.
Exiger une re-authentification forte (step-up authentication) pour toute action critique ou sensible (ex: modification de données RH importantes).
Implémenter une API Gateway avec validation des requêtes (headers, origine, schéma) pour bloquer les requêtes forgées.

MITIGATION À APPLIQUER
Implémenter un mécanisme d’échappement systématique de toutes les données en sortie affichées dans l'interface utilisateur React, en fonction du contexte HTML, JavaScript ou URL.
Valider et filtrer strictement toutes les entrées utilisateur côté serveur (backend Spring Boot) et côté client (frontend React) pour supprimer les caractères spéciaux ou le code potentiellement malveillant.
Implémenter une politique de sécurité de contenu (CSP) stricte pour restreindre les sources de scripts, images et autres ressources, empêchant l’exécution de scripts non autorisés.
Déployer un WAF avec des règles de détection et de blocage des attaques XSS.
Activer les attributs de sécurité des cookies (HttpOnly, Secure, SameSite) pour empêcher l'accès aux cookies de session via JavaScript et leur transmission sur des canaux non sécurisés.
Implémenter une validation stricte des chemins de fichiers basée sur une liste blanche (allowlist) de répertoires et de noms de fichiers autorisés.
Stocker les fichiers sensibles, y compris les documents internes, en dehors du répertoire web ou des répertoires accessibles par le backend et les servir via un mécanisme contrôlé.
Appliquer le principe du moindre privilège sur les accès fichiers, en restreignant les droits d’accès du service applicatif aux seuls fichiers nécessaires.
Utiliser des identifiants indirects (UUID) au lieu de chemins de fichiers directs dans les requêtes des utilisateurs.
Journaliser toutes les tentatives d’accès aux fichiers et surveiller les accès aux fichiers critiques.
Déployer une protection anti-DDoS en amont du réseau pour absorber le trafic malveillant avant qu'il n'atteigne l'application.
Implémenter un rate limiting strict par IP, endpoint et utilisateur sur l'API REST du backend et les interactions avec le chatbot.
Déployer un WAF avec protection anti-DDoS applicatif pour filtrer les requêtes malveillantes au niveau de l'application.
Utiliser l'auto-scaling dynamique pour adapter les ressources du backend en fonction de la charge, mais avec des limites pour éviter une explosion des coûts en cas d'attaque.
Mettre en place un load balancing distribué pour répartir le trafic sur plusieurs instances de services.

MITIGATION À APPLIQUER
Valider strictement toutes les entrées (type, format, taille) à chaque point d'entrée de l'application (frontend, backend, chatbot).
Implémenter des schémas de validation (ex: JSON Schema pour les APIs REST, validation des prompts) et rejeter toute donnée non conforme.
Déployer un WAF avec détection des payloads malformés ou anormaux.
Ne jamais exposer d’informations sensibles dans les messages d’erreur de l'application; fournir des messages génériques et journaliser les détails en interne.
Surveiller les logs d’erreurs applicatives (exceptions, crashs) pour détecter les tentatives de fuzzing et les comportements anormaux.
S’assurer que le front-end (proxy, WAF, load balancer) et le back-end Spring Boot utilisent la même interprétation des en-têtes HTTP (Content-Length, Transfer-Encoding).
Rejeter toute requête mal formée ou incohérente et désactiver les comportements HTTP non standards dans les serveurs.
Nettoyer et normaliser les en-têtes HTTP avant traitement par le backend.
Implémenter une validation des sessions et des tokens indépendamment du proxy frontal pour s'assurer que les contrôles de sécurité ne sont pas contournés.
Limiter le nombre de connexions persistantes (keep-alive) et appliquer des timeouts stricts sur les connexions.
Vérifier systématiquement les autorisations côté serveur pour chaque requête accédant à une ressource, en s'assurant que l'utilisateur est bien le propriétaire ou a les droits d'accès.
Ne jamais faire confiance aux identifiants fournis par le client; les mapper à l'identité authentifiée de l'utilisateur.
Utiliser des identifiants non prévisibles comme des UUID v4 pour les ressources, afin d'empêcher l'énumération.
Implémenter un contrôle d'accès basé sur les rôles (RBAC) ou les attributs (ABAC) pour limiter chaque utilisateur aux actions et ressources autorisées selon son rôle.
Journaliser tous les accès aux ressources sensibles et corréler les événements vers le SIEM pour détecter les tentatives d'exploitation IDOR.

MITIGATION À APPLIQUER
Classifier les données sensibles manipulées par l'assistant RH et masquer ou tronquer ces données dans les interfaces et les journaux en production.
Désactiver les messages d’erreur détaillés en production; n'afficher que des messages génériques et logguer les détails en interne.
Supprimer les fichiers de debug, les logs de développement et les backups accessibles publiquement sur les environnements de production.
Sécuriser les fichiers de configuration et les secrets applicatifs (clés API Azure OpenAI, identifiants de base de données) en les stockant dans un coffre-fort de secrets sécurisé (Vault).
Durcir les serveurs web et applicatifs selon des benchmarks de sécurité (CIS Benchmark) et désactiver les bannières techniques et les en-têtes verbeux.
Chiffrer toutes les communications inter-microservices gRPC avec mTLS (mutual TLS) pour garantir la confidentialité et l'intégrité des messages.
Authentifier mutuellement les workloads des microservices pour s'assurer que seuls les services autorisés peuvent communiquer entre eux.
Valider strictement la structure des messages gRPC avec un schéma formel (Protobuf) avant traitement.
Mettre en place des mécanismes anti-replay et de contrôle de séquencement des messages via nonce, horodatage et numérotation de séquence pour empêcher les rejeux ou les exécutions hors séquence.
Appliquer des contrôles d’accès (RBAC/ABAC) sur les flux inter-microservices pour limiter chaque microservice aux types de messages et actions autorisés.
Vérifier systématiquement la signature cryptographique de chaque JWT avant toute utilisation de son contenu.
Refuser tout token avec l'algorithme 'alg:none' et restreindre les algorithmes autorisés à une liste blanche stricte (ex: RS256, ES256).
Stocker les clés de signature JWT de manière sécurisée (Vault, KMS) et ne jamais les hardcoder dans le code de l'application.
Utiliser des tokens à courte durée de vie (expiration courte) et des refresh tokens sécurisés gérés côté serveur.

MITIGATION À APPLIQUER
Vérifier les rôles et permissions côté backend, après validation du JWT, et ne jamais faire confiance uniquement aux claims du token pour les décisions d'autorisation.
Forcer l’utilisation de HTTPS (TLS 1.3 minimum) pour toutes les communications entre le frontend et le backend.
Implémenter mTLS (mutual TLS) pour toutes les communications gRPC entre microservices afin d'authentifier mutuellement chaque partie.
Valider strictement les certificats côté client et serveur pour s'assurer de l'identité des interlocuteurs et éviter les faux certificats.
Mettre en place HSTS (HTTP Strict Transport Security) pour empêcher les rétrogradations vers HTTP non chiffré.
Utiliser des suites cryptographiques robustes et désactiver les protocoles faibles (SSL, TLS 1.0, TLS 1.1) et les suites de chiffrement obsolètes.
Restreindre strictement les 'redirect_uri' à une liste blanche prédéfinie et refuser toute URL dynamique ou non enregistrée.
Utiliser uniquement HTTPS pour tous les endpoints OAuth afin de garantir le chiffrement des tokens en transit.
Ne jamais exposer les tokens d'accès dans l’URL (query string) et les gérer via des mécanismes sécurisés (ex: HttpOnly cookies ou stockage côté serveur).
Utiliser des tokens courts (short-lived access tokens) et des refresh tokens sécurisés côté serveur uniquement.
Implémenter le paramètre 'state' dans les requêtes OAuth pour prévenir les attaques CSRF.
Générer des tokens de réinitialisation longs, aléatoires et imprévisibles, et les associer à un utilisateur unique côté serveur avec une courte durée de validité.
Implémenter un flux de réinitialisation strict avec des étapes obligatoires et empêcher le contournement de ces étapes.
Ne jamais exposer les tokens de réinitialisation dans les réponses API ou les logs de l'application.
Exiger une vérification supplémentaire (MFA) pour les réinitialisations de mot de passe des comptes sensibles.

MITIGATION À APPLIQUER
Notifier l’utilisateur par un canal secondaire (ex: e-mail) lors d’une demande de réinitialisation et après un changement de mot de passe réussi.
Implémenter un rate limiting sur les demandes de réinitialisation de mot de passe par utilisateur et par adresse IP.
Utiliser des transactions atomiques pour toutes les opérations critiques sur les données de l'assistant RH, en garantissant l'intégrité (ACID).
Implémenter des mécanismes de verrouillage (optimistic locking ou verrous distribués si nécessaire) sur les ressources sensibles pour empêcher les modifications concurrentes.
Centraliser les opérations sensibles dans un service unique ou utiliser des files de traitement (queues) pour sérialiser les opérations critiques et éviter les traitements parallèles non maîtrisés.
Implémenter des mécanismes anti-replay pour les requêtes critiques, garantissant qu'une action n'est exécutée qu'une seule fois.
Journaliser toutes les opérations critiques de création et de modification pour avoir une traçabilité en cas d'incident.
Implémenter une liste blanche (allowlist) stricte des URLs et des domaines autorisés pour les requêtes sortantes initiées par le backend.
Bloquer tout trafic sortant non nécessaire vers Internet (egress deny) et implémenter un pare-feu interne pour bloquer les appels vers le localhost et le réseau interne critique.
Ne jamais permettre à l’utilisateur de contrôler directement une URL ou un nom de domaine dans les entrées de l'application.
Segmenter le réseau entre l'application, les bases de données et les services internes pour limiter la portée d'une attaque SSRF.
Appliquer le principe du moindre privilège sur les rôles IAM attachés aux instances du backend et aux comptes de service, et utiliser des credentials temporaires.
Séparer strictement les données utilisateur de la logique de template; ne jamais construire des templates dynamiquement avec des entrées utilisateur.
Utiliser des templates statiques prédéfinis et passer les données utilisateur uniquement comme variables du template, en échappant ou neutralisant les caractères spéciaux du moteur de template.

MITIGATION À APPLIQUER
Valider strictement toutes les entrées utilisateur avant de les utiliser dans des templates côté serveur.
Activer le mode sandbox du moteur de template si disponible, pour limiter l’accès aux fonctions dangereuses du système d'exploitation ou du système de fichiers.
Exécuter l'application avec des privilèges minimaux et empêcher l’accès aux variables d’environnement sensibles depuis le contexte du template.
Régénérer obligatoirement l'identifiant de session après chaque authentification réussie de l'utilisateur.
Utiliser exclusivement HTTPS pour toutes les communications afin de protéger les cookies de session en transit.
Stocker les identifiants de session uniquement dans des cookies sécurisés avec les flags HttpOnly, Secure et SameSite=Strict.
Invalider immédiatement la session en cas de déconnexion, de changement de mot de passe ou de détection d'une anomalie.
Limiter la durée maximale de validité des sessions pour réduire la fenêtre d'opportunité d'une attaque de fixation.
Imposer une authentification multifacteur (MFA) robuste et résistante au phishing (ex: FIDO2 / WebAuthn) pour tous les employés accédant à l'assistant RH.
Exiger une authentification renforcée (step-up authentication) avant toute action sensible ou consultation de données RH critiques.
Ne jamais dériver l’identité de l’utilisateur depuis le prompt du chatbot, la mémoire de conversation, le contenu RAG ou un output d’outil; toujours résoudre l’identité depuis un jeton/session signé(e) par l’IdP.
Utiliser des cookies HttpOnly + Secure + SameSite=Strict et des sessions courtes avec régénération de l'ID après login.
Implémenter l'Authorization Code + PKCE (Proof Key for Code Exchange) si OAuth/OIDC est utilisé pour les flux d'authentification.
Appliquer un rate limiting par compte, IP, session et endpoint d’authentification.
Déclencher un account lockout temporaire ou un step-up MFA après N échecs d'authentification.

MITIGATION À APPLIQUER
Imposer une authentification multifacteur (MFA) robuste sur les flux de login, reset et recovery.
Générer tous les tokens et session IDs avec un CSPRNG et exiger une entropie suffisante pour tous les artefacts d’authentification.
Vérifier systématiquement la signature cryptographique avant toute acceptation d'un token JWT et valider l’algorithme côté serveur via une liste blanche stricte.
Vérifier côté serveur que l'identifiant de ressource appartient bien à l'utilisateur authentifié du jeton JWT, à chaque appel de l'API REST.
Générer les identifiants de ressources en UUID v4 aléatoires pour empêcher l'énumération prévisible des objets.
Implémenter des contrôles d’accès par rôles (RBAC) ou par attributs (ABAC) pour limiter chaque utilisateur aux actions autorisées selon son rôle sur les objets.
Isoler les microservices par domaine de données pour qu'un service n'expose que ses propres ressources et appliquer une segmentation réseau L3 entre microservices.
Ajouter à l’API Gateway des contrôles de cohérence sur les identifiants d’objet et le contexte d’appel, sans remplacer l’autorisation objet côté service.
Définir des DTOs (Data Transfer Objects) distincts par endpoint de l'API REST, exposant uniquement les propriétés strictement nécessaires dans chaque réponse.
Valider le payload entrant contre un schéma JSON strict avec `additionalProperties: false` pour refuser toute propriété non autorisée dans les requêtes de modification.
Restreindre la modification aux seules propriétés explicitement autorisées pour le rôle de l'appelant via des contrôles d'autorisation granulaires.
Mapper explicitement les champs autorisés vers le modèle métier et désactiver tout binding automatique non maîtrisé par le framework Spring Boot.
Auditer les réponses API avec un outil de 'diff' à chaque déploiement pour détecter les propriétés exposées par inadvertance.
Valider strictement la taille et le format de toutes les entrées utilisateur (input length validation) et des données traitées, en implémentant des limites de buffer explicites.

MITIGATION À APPLIQUER
Utiliser des langages et des frameworks qui offrent des protections contre les manipulations de buffer (comme Java avec Spring Boot) et des fonctions sécurisées pour la manipulation de la mémoire.
Ne jamais exposer les erreurs mémoire ou les dumps au client; journaliser ces informations en interne et protéger les logs.
Exécuter les services de l'application avec des comptes non privilégiés et appliquer le principe du moindre privilège sur les processus.
Activer les protections système au niveau de l'OS (DEP/NX pour la prévention d'exécution de données, ASLR pour la randomisation de l'espace d'adressage).
Éviter l'utilisation de mécanismes de désérialisation natifs non sécurisés (comme la sérialisation Java standard) pour les données non fiables provenant du client.
Valider et nettoyer les données sérialisées provenant du client, et implémenter une allowlist stricte des classes autorisées à la désérialisation.
Désactiver les fonctionnalités dangereuses de désérialisation (auto-type, polymorphisme dynamique, résolution automatique de classes) et configurer les frameworks pour utiliser des modes stricts.
Signer toutes les données sérialisées (HMAC ou signature numérique) pour vérifier leur intégrité avant toute désérialisation.
Isoler les composants traitant des données sérialisées dans des environnements sécurisés et appliquer le principe du moindre privilège à ces services.
Implémenter une allowlist stricte des fichiers autorisés à l'inclusion, et utiliser un mapping interne au lieu d'un chemin fourni par l'utilisateur.
Interdire toute inclusion de fichiers distants et centraliser les fichiers inclus dans un répertoire dédié et contrôlé.
Valider strictement tous les paramètres liés aux fichiers et empêcher l'accès aux fichiers sensibles (config, logs, clés, système).
Stocker les fichiers sensibles hors des répertoires accessibles par le serveur web ou l'application.
Appliquer le principe du moindre privilège sur les fichiers et restreindre les droits d'accès du service applicatif aux seuls fichiers nécessaires.

MITIGATION À APPLIQUER
Journaliser les accès aux fichiers et surveiller l’intégrité des fichiers critiques.
Restreindre l’accès réseau aux registres de packages, artefacts et aux dépôts de documents internes aux services internes autorisés uniquement.
Implémenter un IAM centralisé avec le principe du moindre privilège pour l’accès aux dépôts de code, registres d’artefacts, datasets (documents internes) et pipelines RAG.
Vérifier l’intégrité cryptographique (hachage SHA-256) des documents internes et des embeddings locaux avant leur utilisation ou déploiement.
Maintenir un AI Bill of Materials (AI-BOM) pour les modèles, datasets (documents internes), dépendances et adaptateurs utilisés dans le pipeline RAG.
Scanner les dépendances directes et transitives (SCA) utilisées dans le backend et le pipeline RAG pour détecter les vulnérabilités.
Implémenter un contrôle d’accès strict (IAM) basé sur le principe du moindre privilège pour l’accès aux datasets (documents internes) et aux bases de données pgvector.
Appliquer un mécanisme de hachage (SHA-256) sur les documents internes et les embeddings afin de vérifier leur intégrité avant chaque phase d’entraînement du RAG ou d'utilisation.
Maintenir une traçabilité complète (dataset provenance) des documents internes, incluant la source et l'historique des modifications.
Mettre en place des mécanismes de File Integrity Monitoring (FIM) pour détecter toute modification non autorisée des documents internes et des embeddings.
Détection automatique d’outliers ou d'anomalies dans les datasets de documents internes pour identifier des tentatives d'empoisonnement.
Chiffrer les datasets d’entraînement (documents internes) et les embeddings au repos dans pgvector (AES-256).
Stocker le 'system prompt' uniquement côté backend et empêcher toute modification ou divulgation de celui-ci par les utilisateurs ou le LLM lui-même.
Séparer clairement les instructions système (garde-fous) et les données utilisateur dans le prompt envoyé au LLM, en appliquant une hiérarchie stricte.

MITIGATION À APPLIQUER
Implémenter une LLM Security Gateway ou AI Firewall pour inspecter et filtrer tous les prompts avant envoi au modèle LLM externe, bloquant les tentatives d'injection.
Limiter la longueur maximale des prompts utilisateur et supprimer ou neutraliser les balises HTML, Markdown et caractères spéciaux qui pourraient être interprétés comme des instructions.
Déployer des 'guardrails' pour contrôler les réponses du modèle et empêcher la divulgation d'informations sensibles ou de secrets.
Valider et filtrer toutes les données externes (documents internes) avant leur ingestion et indexation dans pgvector.
Nettoyer les documents en supprimant les scripts, le HTML actif et le contenu dynamique avant l'embedding et l'indexation.
Implémenter un pipeline de sanitization robuste avant l'indexation dans pgvector pour détecter et neutraliser les instructions cachées ou le contenu malveillant.
Mettre en place un contrôle d’accès strict pour l’ingestion et la modification des documents internes utilisés par la RAG.
Utiliser une architecture multi-LLM ou un modèle secondaire ('LLM-as-a-judge') pour analyser les entrées RAG et les réponses du modèle, et détecter les tentatives d'injection.
Implémenter une authentification et autorisation forte (AuthN/AuthZ) sur chaque opération de lecture/écriture dans la base de données pgvector.
Valider l'intégrité de la mémoire à long terme (embeddings) à l'aide de lignes de base cryptographiques (hachage SHA-256) et surveiller les sorties dérivées du retrieval pour détecter un chunk empoisonné.
Sanitiser le contenu des documents avant indexation et avant injection dans le contexte du LLM: suppression de patterns d’injection, filtrage de markup suspect.
N’autoriser l’ingestion des documents internes dans pgvector que depuis des sources approuvées et avec une provenance vérifiée (historique, métadonnées, confiance de la source).
Chiffrer les embeddings stockés dans pgvector au repos et en transit, surtout s'ils contiennent des données sensibles.
Versionner les documents sources, les embeddings et les transformations pour permettre la comparaison, la détection de dérive et le rollback en cas de corruption.

MITIGATION À APPLIQUER
Déployer une API Gateway pour appliquer des quotas, du throttling et un rejet précoce des requêtes excessives ou coûteuses vers l'API Azure OpenAI et les services RAG.
Limiter la taille maximale des prompts et des documents utilisés pour la RAG et rejeter les requêtes dépassant la fenêtre de contexte autorisée.
Définir des timeouts stricts pour les requêtes d'inférence au LLM et les appels aux composants RAG.
Implémenter un autoscaling borné des workers d’inférence du backend de l'assistant RH et des processus RAG.
Journaliser le nombre de requêtes, la latence, les erreurs et la taille des prompts pour détecter les hausses anormales de consommation ou les patterns d’abus.
Implémenter un mode dégradé en cas de surcharge, comme la réduction du contexte RAG ou le retour à des réponses pré-générées pour les requêtes non critiques.
Valider strictement chaque entrée (prompt) avant inférence, en n’acceptant que les formats d’entrée attendus et en rejetant les requêtes mal formées ou incohérentes.
Détecter les entrées hors distribution attendue ou les anomalies sur les requêtes d’inférence avant soumission au modèle LLM externe.
Implémenter un AI Firewall ou une LLM Security Gateway pour inspecter les requêtes d’inférence et filtrer les entrées malveillantes.
Limiter les informations retournées par l’API du LLM au strict nécessaire et ne pas exposer les scores, probabilités ou détails inutiles de sortie qui pourraient aider à l'évasion.
Mettre en place une seconde validation ou un seuil de confiance pour les décisions critiques du LLM, avec une intervention humaine si nécessaire.
Déployer les services d’inférence du backend derrière une API Gateway sécurisée qui applique un throttling adaptatif pour détecter les requêtes automatisées ou excessives.
Restreindre l’accès réseau aux registres de modèles (si applicables) et aux bases de données pgvector aux services internes autorisés uniquement.
Limiter les informations retournées par les APIs d’inférence du LLM au strict nécessaire, en réduisant la granularité des scores de prédiction exposés.

MITIGATION À APPLIQUER
Chiffrer les embeddings locaux stockés dans pgvector au repos et en transit.
Journaliser toutes les requêtes d’inférence au LLM et les accès à pgvector, et surveiller les activités suspectes (pics de requêtes, accès inhabituels).
Implémenter un IAM strict avec RBAC granulaire pour contrôler l’accès aux documents internes et aux bases de données pgvector (train, upload, update, deploy, rollback).
Appliquer une signature numérique obligatoire sur les documents internes et les embeddings avant leur indexation/déploiement, et vérifier leur intégrité via hachage cryptographique (SHA-256) avant chaque utilisation.
Chiffrer les documents internes et les embeddings stockés dans pgvector au repos avec AES-256.
Implémenter un versioning strict des documents et des embeddings via un système de gestion de données pour permettre un rollback rapide vers une version saine en cas de compromission.
Mettre en place une validation avec double approbation pour toute modification majeure des documents internes ou des embeddings.
Séparer physiquement les buffers mémoire en zones d'instructions, de travail et de données externes dans le traitement des requêtes du chatbot.
Appliquer un schéma de validation strict (JSON Schema ou validation sémantique) sur toute sortie d'outil ou de composant RAG avant son intégration en mémoire de travail du chatbot.
Ré-injecter systématiquement les règles de sécurité au sommet du contexte de conversation après chaque phase de compression pour les 'hard-pinner'.
Extraire des faits structurés (plutôt que des résumés narratifs) des interactions pour limiter la ré-interprétation malveillante des instructions par le LLM.
Utiliser un modèle tiers (Critique) pour valider que le résumé est sémantiquement fidèle aux logs bruts et exempt d'instructions malveillantes.